

LAPORAN PRAKTIKUM 2
PEMOGRAMAN BERORIENTASI OBJEK



Disusun Oleh:

NIDA TSABITA Ariba

2511532025

Praktikum Kelas A

Dosen Pengampu:

NURFIAH, S.ST, M.Kom.

Asisten Praktikum:

MUHAMMAD DAWI SYAUQI

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

A. Tahapan Pelaksanaan Praktikum

Pada praktikum ini, program *Minibank* dikembangkan dengan menambahkan fitur untuk menyimpan riwayat transaksi. Setiap transaksi setor dan tarik tunai akan dicatat dan disimpan ke dalam *ArrayList*. Dengan adanya fitur ini, pengguna dapat melihat kembali transaksi yang telah dilakukan.

1. Membuat *Class Transaksi*

Class Transaksi digunakan untuk menyimpan informasi dari setiap transaksi yang dilakukan.

1.1 Atribut *Class Transaksi*

```
String idTransaksi;  
String jenis;  
double nominal;
```

Atribut *idTransaksi* digunakan untuk menyimpan ID transaksi, *jenis* digunakan untuk menentukan jenis transaksi seperti Kredit atau Debit, sedangkan *nominal* digunakan untuk menyimpan jumlah uang yang ditransaksikan.

1.2 Constructor *Class Transaksi*

```
public Transaksi(String id, String jenis, double nominal) {  
    this.idTransaksi = id;  
    this.jenis = jenis;  
    this.nominal = nominal;  
}
```

Constructor digunakan untuk mengisi data awal ketika objek *Transaksi* dibuat. Nilai dari parameter *id*, *jenis*, dan *nominal* akan dimasukkan ke masing-masing atribut.

1.3 Method *cetakDetail()*

```
public void cetakDetail() {  
    System.out.println("ID: " + idTransaksi + " | Jenis:  
" + jenis + " | Nominal: Rp" + nominal);  
}
```

Method cetakDetail() digunakan untuk menampilkan informasi transaksi berupa ID, jenis transaksi, dan nominal.

2. Menambahkan Relasi Riwayat Transaksi pada *Class Rekening*

Pada *class Rekening*, ditambahkan *ArrayList* untuk menyimpan semua transaksi yang dilakukan oleh rekening.

2.1 Atribut Riwayat Transaksi

```
ArrayList<Transaksi> riwayatTransaksi;
```

riwayatTransaksi digunakan untuk menyimpan objek-objek Transaksi. Jadi, satu rekening dapat memiliki banyak data transaksi.

2.2 Inisialisasi pada *Constructor Rekening*

```
public Rekening(String nomor, String nama, double saldoAwal) {  
    nomorRekening = nomor;  
    namaPemilik = nama;  
    saldo = saldoAwal;  
  
    this.riwayatTransaksi = new ArrayList<>();  
  
    System.out.println("Rekening atas nama " +  
namaPemilik + " berhasil di buat dengan saldo Rp" + saldo);  
}
```

Pada saat objek Rekening dibuat, *riwayatTransaksi* juga dibuat menggunakan *new ArrayList<>()*. Hal ini dilakukan agar *ArrayList* siap digunakan untuk menyimpan transaksi.

3. Menambahkan Pencatatan Transaksi pada *setorTunai*

Method setorTunai() digunakan untuk melakukan penyetoran uang sekaligus mencatat transaksi tersebut.

```
public void setorTunai(double nominal) {  
    if (nominal > 0) {  
        saldo += nominal;  
  
        String idTrx = "TRX-S-" +  
System.currentTimeMillis();  
        Transaksi trxBaru = new Transaksi(idTrx,  
"Kredit", nominal);  
        riwayatTransaksi.add(trxBaru);  
  
        System.out.println("Setor tunai Rp" + nominal  
+ " berhasil. Saldo saat ini: Rp" + saldo);  
    } else {  
        System.out.println("Gagal: Nominal setor  
harus lebih dari 0!");  
    }  
}
```

Program terlebih dahulu mengecek apakah nominal setoran lebih dari 0. Jika benar, saldo akan bertambah. Setelah itu, program membuat ID transaksi, membuat objek Transaksi dengan jenis Kredit, kemudian menyimpannya ke dalam *riwayatTransaksi*.

B. Latihan

1. Integrasi Rekam Jejak Penarikan

Pada tugas ini, *method tarikTunai()* ditambahkan proses untuk mencatat transaksi penarikan ke dalam riwayat transaksi.

```
public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println("Transaksi Gagal: Minimal nominal penarikan Rp10.000");
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
    } else {
        saldo -= nominal;

        String idTrx = "TRX-T-" +
            System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx,
            "Debit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Tarik tunai berhasil! saldo anda tersisa: Rp" + saldo);
    }
}
```

Sebelum melakukan penarikan, program mengecek dua kondisi. Pertama, nominal penarikan harus minimal Rp10.000. Kedua, jumlah penarikan tidak boleh lebih besar dari saldo. Jika kedua kondisi terpenuhi, saldo akan berkurang. Selanjutnya, program membuat ID transaksi dengan awalan "TRX-T-", membuat objek Transaksi dengan jenis Debit, dan menyimpannya ke dalam *riwayatTransaksi*.

2. Fitur Cetak Mutasi Rekening

Pada tugas ini dibuat *method cetakMutasi()* untuk menampilkan riwayat transaksi yang telah dilakukan.

```
public void cetakMutasi() {
    if (riwayatTransaksi.isEmpty()) {
```

```

        System.out.println("Belum ada transaksi pada
rekening ini");
    } else {
        System.out.println("Mutasi Rekening");
        int total = riwayatTransaksi.size();
        int startIndex = (total > 3) ? (total - 3) :
0;

        for (int i = startIndex; i < total; i++) {
            riwayatTransaksi.get(i).cetakDetail();
        }
    }
}

```

Method cetakMutasi() digunakan untuk menampilkan riwayat transaksi yang tersimpan pada *riwayatTransaksi*. Pertama, program mengecek apakah terdapat transaksi. Jika belum ada transaksi, program akan menampilkan pesan "Belum ada transaksi pada rekening ini". Jika terdapat transaksi, program akan menampilkan seluruh data transaksi yang tersimpan.

Variabel *total* digunakan untuk mengetahui jumlah transaksi yang ada. Selanjutnya, perulangan *for* digunakan untuk mengambil setiap transaksi dari *riwayatTransaksi*, kemudian menampilkan detail transaksi menggunakan *method cetakDetail()*.

Penambahan Menu pada *class Main*

```

case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka
rekening!");
    } else {
        akunAktif.cetakMutasi();
    }
    break;

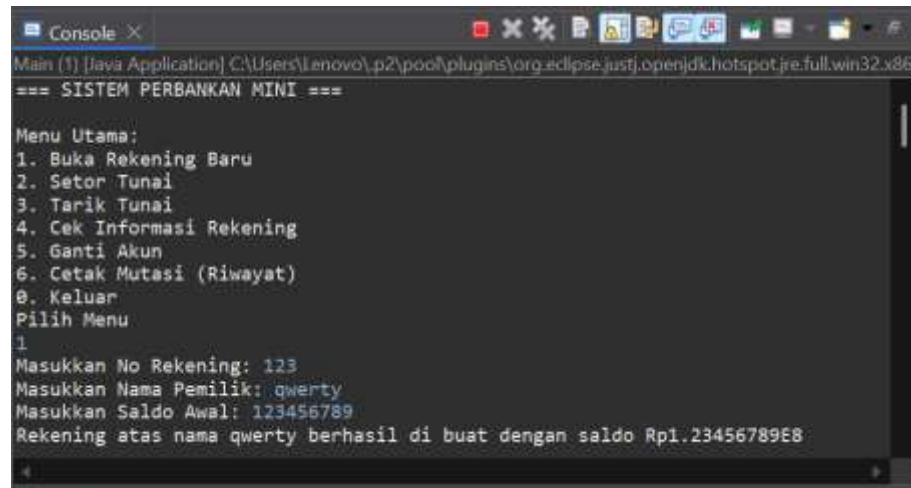
```

Pada *class Main*, ditambahkan menu 6 untuk melihat mutasi rekening. Program mengecek terlebih dahulu apakah ada akun yang aktif. Jika *akunAktif* masih *null*, berarti belum ada rekening yang dipilih sehingga program akan menampilkan *Error*. Jika akun sudah aktif program akan menjalankan *cekMutasi()* untuk menampilkan riwayat transaksi.

C. Hasil Praktikum

1. Membuat rekening 1

Pada pengujian pertama, program digunakan untuk membuat rekening baru dengan memasukkan nomor rekening, nama pemilik, dan saldo awal. Program berhasil membuat rekening atas nama qwerty dengan nomor rekening 123 dan saldo awal sebesar Rp123.456.789.

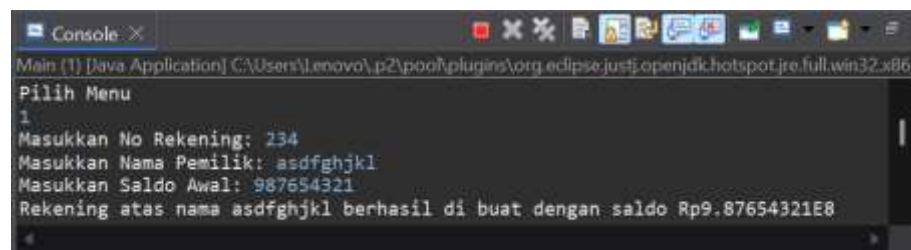


```
Console X
Main (1) [Java Application] C:\Users\Lenovo\p2\poo\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih Menu
1
Masukkan No Rekening: 123
Masukkan Nama Pemilik: qwerty
Masukkan Saldo Awal: 123456789
Rekening atas nama qwerty berhasil di buat dengan saldo Rp1.23456789E8
```

2. Membuat rekening 2

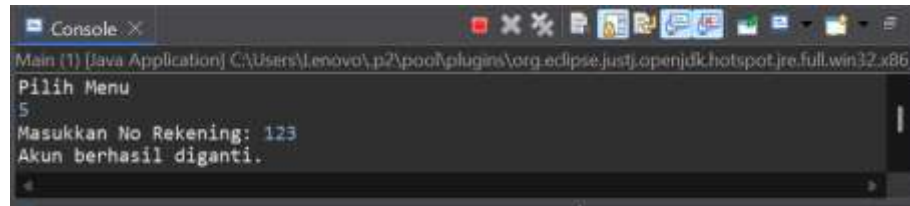
Selanjutnya, dibuat rekening kedua dengan nomor rekening 234, nama pemilik asdfghjkl, dan saldo awal sebesar Rp987.654.321. Kedua rekening berhasil disimpan oleh program.



```
Console X
Main (1) [Java Application] C:\Users\Lenovo\p2\poo\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86
Pilih Menu
1
Masukkan No Rekening: 234
Masukkan Nama Pemilik: asdfghjkl
Masukkan Saldo Awal: 987654321
Rekening atas nama asdfghjkl berhasil di buat dengan saldo Rp9.87654321E8
```

3. Ganti akun

Setelah dua rekening dibuat, dilakukan pengujian fitur Ganti Akun. Nomor rekening 123 dimasukkan untuk memilih rekening atas nama qwerty. Program menampilkan pesan "Akun berhasil diganti.", yang menunjukkan bahwa rekening berhasil menjadi akun aktif.



```
Console X
Main (1) [Java Application] C:\Users\Lenovo\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86
Pilih Menu
5
Masukkan No Rekening: 123
Akun berhasil diganti.
```

4. Setor tunai

Setelah rekening 123 menjadi akun aktif, dilakukan setor tunai setiap transaksi setor tunai dicatat sebagai transaksi dengan jenis Kredit pada riwayat transaksi.



```
Console X
Main (1) [Java Application] C:\Users\Lenovo\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86
Pilih Menu
2
Masukkan nominal setor: Rp123456789
Setor tunai Rp1.23456789E8 berhasil. Saldo saat ini: Rp2.46913578E8
```

5. Tarik tunai

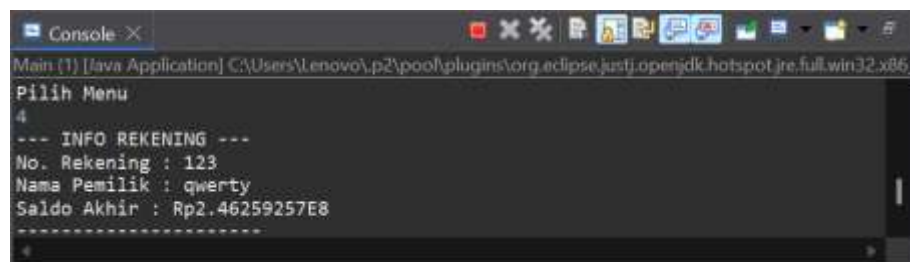
Selanjutnya dilakukan penarikan tunai, dan transaksi tersebut kemudian dicatat dalam riwayat dengan jenis Debit.



```
Console X
Main (1) [Java Application] C:\Users\Lenovo\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86
Pilih Menu
3
Masukkan nominal tarik: Rp654321
Tarik tunai berhasil! saldo anda tersisa: Rp2.46259257E8
```

6. Cek informasi rekening

Menu Cek Informasi Rekening digunakan untuk melihat data rekening yang sedang aktif. Program menampilkan nomor rekening 123, nama pemilik qwerty, dan saldo yang tertera

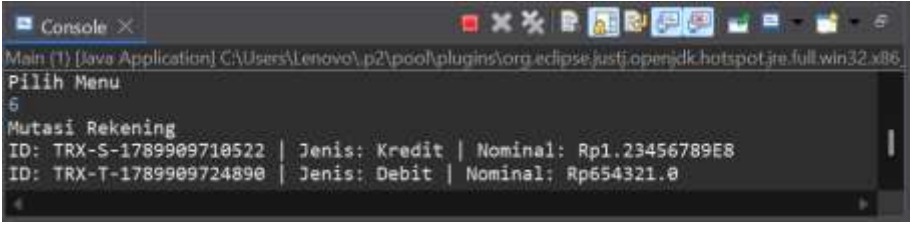


```
Console X
Main (1) [Java Application] C:\Users\Lenovo\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86
Pilih Menu
4
--- INFO REKENING ---
No. Rekening : 123
Nama Pemilik : qwerty
Saldo Akhir : Rp2.46259257E8
-----
```

7. Cetak mutasi

Terakhir, dilakukan pengujian menu Cetak Mutasi. Program berhasil menampilkan transaksi yang telah dilakukan, setiap transaksi menampilkan ID transaksi, jenis transaksi, dan nominal transaksi.

Transaksi setor ditampilkan dengan jenis Kredit, sedangkan transaksi tarik ditampilkan dengan jenis Debit.



```
Console X
Main (1) [Java Application] C:\Users\Lenovo\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64
Pilih Menu
5
Mutasi Rekening
ID: TRX-S-1789909710522 | Jenis: Kredit | Nominal: Rp1.23456789E8
ID: TRX-T-1789909724890 | Jenis: Debit | Nominal: Rp54321.0
```

D. Link Repository GitHub

<https://github.com/aaribbb/PrakPBO2026> B 2511532025